Git and GitHub

Handbook

A Complete Guide to Version Control, Collaboration and Modern Development



GIT AND GITHUB

1. What is Git ?

Ans:- Git is a distributed version control system used to track changes in source code, keep a history of work, and help developers collaborate.

2. Why is Git used ?

Ans:- Git is used because it helps to:

- Track code changes.
- Collaborate with others.
- Maintain versions.
- Back up project history.
- Restore earlier versions.

3. Key features of Git ?

Ans:- Some key features of Git are:

- Distributed version control system.
- Fast and efficient.
- Branching and merging.
- Commit history.
- Works offline.

4. What is GitHub ?

Ans:- GitHub is an online platform for hosting Git repositories, sharing code, reviewing changes, and collaborating on projects.

5. Git vs GitHub ?

Ans:- Git is the version control tool that tracks changes locally. GitHub hosts Git repositories online and adds collaboration features.

GIT BASICS

6. What is a repository ?

Ans:- A repository (repo) stores project files and their version history.

7. What is a commit ?

Ans:- A commit is a saved snapshot of changes with a message describing the update.

8. What is a branch ?

Ans:- A branch is an independent line of development used to work on a feature without changing the main branch.

9. What is the working directory ?

Ans:- It is the folder containing the files you are currently editing.

10. What is the staging area ?

Ans:- It holds selected changes that will be included in the next commit.

11. What is a remote repository ?

Ans:- A repository hosted elsewhere, often on GitHub, that you can connect to and share with.

INSTALLING GIT

19. How to install Git on Windows ?

Ans:- 1. Go to the official Git website.
2. Download the installer for Windows.
3. Run the installer and follow the steps.
4. Verify the installation using git --version.

20. How to install Git on macOS ?

Ans:- 1. Open Terminal.
2. Use Homebrew command:

```
brew install git
```

3. Verify using git --version.

21. How to install Git on Linux ?

Ans:- Use the package manager of your distribution.

- For Ubuntu/Debian:

```
sudo apt install git
```

- For CentOS/RHEL:

```
sudo yum install git
```

- For Fedora:

```
sudo dnf install git
```

22. How to verify Git installation ?

Ans:- Open terminal or command prompt and run:

```
git --version
```

It will display the installed Git version.

CONFIGURING GIT

23. What is Git configuration ?

Ans:- Git configuration is used to set custom settings like your name, email, editor, and default preferences.

24. How to set your name and email ?

Ans:- Use the following commands:

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

25. How to check current configuration ?

Ans:- Use the command:

```
git config --list
```

This will display all configured settings.

26. How to set a default code editor ?

Ans:- You can set your preferred editor using:

```
git config --global core.editor "code"
```

You can use "notepad", "vim" or any other editor.

27. How to configure line ending behaviour ?

Ans:- Git handles line endings differently on different operating systems. You can set it using:

```
git config --global core.autocrlf true
```

This helps avoid line ending issues in cross-platform projects.

GIT COMMANDS

12. Command to initialize Git ?
Ans:- git init - creates a new local Git repository.

13. Command to check repository status ?
Ans:- git status - shows changed, staged, and untracked files.

14. Command to stage a file ?
Ans:- git add filename - adds a file's changes to the staging area.

15. Command to commit changes ?
Ans:- git commit -m "message" - saves staged changes with a message.

16. Command to view commit history ?
Ans:- git log - displays previous commits.

17. Command to clone a repository ?
Ans:- git clone <repository-url> - copies a remote repository to your computer.

18. Command to download and merge updates ?
Ans:- git pull - fetches and integrates changes from a remote repository.

CREATING A REPOSITORY

28. How to create a new repository ?

Ans:- Use the command:

```
git init
```

This creates a new empty Git repository in your current directory.

29. How to check repository status ?

Ans:- Use the command:

```
git status
```

It shows the current state of your working directory and staging area.

30. How to add files to the staging area ?

Ans:- Use the command:

```
git add <filename>
```

To add all files, use:

```
git add .
```

31. How to commit changes ?

Ans:- Use the command:

```
git commit -m "your message"
```

This saves the staged changes with a message.

32. How to view commit history ?

Ans:- Use the command: `git log`

It displays the list of previous commits.

GITHUB ORGANIZATIONS

162. What is a GitHub Organization ?

Ans:- A GitHub Organization is an account that allows multiple people to work together on many repositories under a single organization. It is useful for teams and companies.

163. Why use a GitHub Organization ?

Ans:- It helps to:

- Manage multiple repositories.
- Control access and permissions.
- Collaborate with a large team.
- Keep personal and work projects separate.

164. How to create an Organization ?

Ans:-
1. Go to https://github.com/organizations/new
2. Enter the organization name.
3. Choose a plan (free or paid).
4. Click "Create organization".

165. What are roles in an Organization ?

Ans:- GitHub provides different roles with different permissions:

- Owner - Full access and control.
- Member - Contribute to repositories.
- Moderator - Manage discussions and members.
- Billing Manager - Manage billing and plans.

166. How to invite a member to an Organization ?

Ans:-
1. Go to your organization on GitHub.
2. Click on "People" → "Invite member".
3. Enter the GitHub username or email.
4. Choose the role and click "Send invitation".

GITHUB PERMISSIONS AND ACCESS CONTROL

167. What is access control in GitHub ?

Ans:- Access control allows you to define who can view, edit, or manage your repositories. It helps to keep your code secure and organized.

168. Types of access in a repository ?

Ans:- GitHub provides the following access levels:

- Read - Can view and clone the repository.
- Triage - Can manage issues and pull requests.
- Write - Can push code to the repository.
- Maintain - Can manage repository settings.
- Admin - Full access including sensitive settings.

169. How to manage access for a repository ?

Ans:-
1. Go to your repository on GitHub.
2. Click on "Settings".
3. Go to "Collaborators and teams".
4. Add people or teams and choose the access level.

170. What is a team in GitHub ?

Ans:- A team is a group of people within an organization. Teams help to manage access to multiple repositories at once.

171. Example of adding a team to a repository :

Ans:-
1. Go to "Settings" → "Collaborators and teams".
2. Click "Add teams".
3. Select the team and choose the permission level.
4. Click "Add team".

GITHUB API AND INTEGRATIONS

172. What is GitHub API ?

Ans:- GitHub API allows developers to interact with GitHub programmatically. It can be used to automate tasks, manage repositories, issues, pull requests and more.

173. Why use GitHub API ?

Ans:- It helps to:

- Automate repetitive tasks.
- Integrate GitHub with other tools.
- Build custom applications.
- Extract data from GitHub.

174. What are the types of GitHub API ?

Ans:-
1. REST API - Uses standard HTTP requests.
2. GraphQL API - Allows flexible and efficient queries.

175. Example of a REST API request :

Ans:-

```
GET https://api.github.com/users/username
Headers:
  Authorization: token <your_token>
  Accept: application/vnd.github.v3+json
```

176. Example of a GraphQL API request :

Ans:-

```
POST https://api.github.com/graphql
Headers:
  Authorization: bearer <your_token>
  Content-Type: application/json
Body:
    {
      "query":"{ viewer { login name } }"
    }
```

GITHUB PROJECTS

177. What is GitHub Projects ?

Ans:- GitHub Projects is a project management tool that helps you plan, track and manage your work using boards, lists, cards and automation. It is useful for individuals and teams to organize tasks related to issues, pull requests and development.

178. Why use GitHub Projects ?

Ans:- It helps to:

- Plan and track tasks.
- Visualize progress using Kanban boards.
- Link issues and pull requests.
- Collaborate with team members.
- Improve project organization.

179. Types of GitHub Projects ?

Ans:- GitHub provides two types of Projects:

1. Classic Projects - Older version (simple boards).
2. Projects (Beta) - New version with more features like automation, custom fields and better integration.

180. How to create a GitHub Project ?

Ans:-

1. Go to your repository or organization.
2. Click on the "Projects" tab.
3. Click "New project".
4. Choose a template (e.g., Kanban, Backlog, etc.).
5. Add columns and start creating cards.

181. Common features in GitHub Projects ?

Ans:- Some useful features are:

- Custom fields (status, priority, assignee, etc.).
- Automation (move card when issue is closed).
- Views (Board, Table, Roadmap).
- Integration with issues and pull requests.

GITHUB CODESPACES

182. What is GitHub Codespaces ?

Ans:- GitHub Codespaces is a cloud-based development environment that allows you to code directly in the browser or your local editor. It provides a fully configured environment with all the tools, dependencies and settings needed for your project.

183. Why use GitHub Codespaces ?

Ans:- It helps to:

- Start coding quickly without local setup.
- Work from any device with a browser.
- Get a consistent development environment.
- Collaborate easily with team members.
- Save time on environment setup.

184. How to create a Codespace ?

Ans:-

1. Go to your repository on GitHub.
2. Click on "Code" button.
3. Select the "Codespaces" tab.
4. Click "Create codespace".
5. Wait for the environment to start.

185. What can you do in a Codespace ?

Ans:-

- Write and run code.
- Install dependencies.
- Use terminal and development tools.
- Commit and push changes to your repository.
- Collaborate in real-time.

186. Things to keep in mind :

Ans:-

- Codespaces may have usage limits (especially on free plans).
- It requires an active internet connection.
- You can customize the environment using a devcontainer.json file.

GITHUB ADVANCED FEATURES

187. What are some advanced features in GitHub ?

Ans:- GitHub provides several advanced features to improve productivity, automation and collaboration. Some of them are

- GitHub Actions
- GitHub Pages
- GitHub Codespaces
- GitHub Projects
- GitHub Security (Dependabot, Code Scanning)
- GitHub Discussions
- GitHub API
- GitHub Webhooks

188. What is GitHub Actions used for ?

Ans:- GitHub Actions is used for CI/CD automation. It allows you to automate tasks like testing, building and deploying your code whenever changes are pushed to the repository.

189. What is Dependabot ?

Ans:- Dependabot is a GitHub feature that automatically checks for outdated dependencies and creates pull requests to update them, helping to keep your project secure.

190. What is Code Scanning ?

Ans:- Code Scanning helps to find security vulnerabilities and coding errors in your code using static analysis.

191. What is a Webhook in GitHub ?

Ans:- A webhook allows GitHub to send real-time notifications to an external service when specific events occur (e.g., push, pull request, issue created).

192. Why are these advanced features important ?

Ans:- They help to:

- Improve code quality and security.
- Automate development workflows.
- Save time and reduce manual work.
- Enhance collaboration and project management.

Index (Table of Contents)

S.No.	Topic	Page No.
1.	Introduction to Git	3
2.	Why Version Control ?	4
3.	Installing Git	5
4.	Basic Git Configuration	6
5.	Git Workflow	7
6.	Creating a Repository	8
7.	Git File States	9
8.	Adding and Committing Files	10
9.	Viewing Commit History	11
10.	Working with Branches	12
11.	Merging Branches	13
12.	Handling Merge Conflicts	14
13.	Working with Remote Repositories	15
14.	Cloning a Repository	16
15.	Pushing and Pulling Changes	17
16.	GitHub Basics	18
17.	Creating a GitHub Account	19
18.	Creating a Repository on GitHub	20
19.	Forking a Repository	21
20.	Pull Requests	22
21.	Issues in GitHub	23
22.	Protecting Branches	24
23.	GitHub Workflows and GitHub Actions	25
24.	Markdown in GitHub	26
25.	GitHub Pages	27
26.	GitHub Profile and README	28
27.	GitHub Collaboration Tools	29
28.	GitHub Security	30
29.	GitHub CLI	31
30.	Git Ignore and Sensitive Files	32
31.	GitHub Repository Settings	33
32.	Deploying Projects with GitHub Pages	34
33.	Real-World Git and GitHub Workflow	35
34.	GitHub Organizations	36
35.	GitHub Permissions and Access Control	37
36.	GitHub API and Integrations	38
37.	GitHub Projects	39
38.	GitHub Codespaces	40
39.	GitHub Advanced Features	41
40.	Best Practices, Common Mistakes and Conclusion	42